Broker Tech: Retail FX and CFD Risk Analytics in kdb+/q

Summary

A retail FX and CFD broker is not really in the business of predicting markets. It is in the business of managing a book of client flow: deciding which clients to hedge with a liquidity provider (A booking them), which clients to take the other side of internally (B booking them), watching for exposure that has gotten too concentrated in one symbol or one client, catching execution behaviour that looks like quote stuffing or latency gaming, and pricing borrow and swap and commission so the desk actually makes money on the flow it keeps. This is the exact business a platform like tapaas.com sells software for, and it is the business `brokerTech` models.

`brokerTech` is a module in openQ, the author's kdb+ platform. It is pure batch analytics, no tp/rdb/cep pipeline, over a real MetaTrader "Signals" copy trading dataset scraped from 2015 to 2026: per provider trade blotters, intraday equity curves, reported monthly returns, and provider identity. Every function is the same shape as `spread.q` and `primeFinance.q` and `candle.q`: a table in, a table out, pure, testable on its own. On top of it sits `openDash`, a Node.js gateway plus React frontend that turns the library into five live dashboard pages: an overview, an A book / B book routing simulator, a toxicity analysis simulator, a client clustering view, and an execution quality drilldown.

This piece walks through the library the way a broker risk desk would actually use it: what exposure and concentration look like on a real book, how execution quality gets scored, how a composite toxicity score gets built out of five explainable components instead of a fitted model, how the A/B routing recommendation turns that into a dollar figure, how a from scratch k means clustering step groups clients by behaviour, and how the frontend turns two of those formulas into live sliders the desk can drag without a round trip to kdb+. Every number quoted below came from running the module's own report against the real archive this session, not from synthetic data, and one specific claim in the module's own README did not survive that check.

Repo

`brokerTech` is not code embedded in this repository. It is a module under `modules/analytics/brokerTech/` in openQ, and its dashboard side lives in a separate repo, `openDash`, under `gateway/src/brokerTech.js` and the React file `openq-dashboard/src/main.jsx`. The kdb+ side has no `-procType` role of its own. `run.q` loads the archive read only and prints a report, the same shape as `modules/backtest/run.q` and `modules/analytics/candle/run.q`. The one process config, `cfg_proc/modules/brokerTech/hdb.json`, is a plain generic `hdb` role fronting the same archive on port 5077, distinguished only by having `brokerTech.q` in its library list so the `.brk.*` namespace is available in process for IPC queries to call directly.

The data and its sign conventions

`schema_retail.q` stubs five tables. In this dataset "client" means signal provider, one `signalId` per copy trading account: `sig` (one row per provider), `trade` (one row per order or ledger entry, partitioned by scrape date), `equity` (intraday balance and equity samples), `growth` (a running growth percentage), and `monthly` (per provider, per month returns as reported to the marketplace, unvalidated).

Getting the sign conventions right once, in one place, matters more here than almost anywhere else in the library, because every downstream revenue number depends on them. `profit` is the client's realised P&L on a trade. `commission` and `swap` are stored negative, meaning a cost to the client, so broker fee and financing revenue is `neg commission` and `neg swap`. If the broker internalises a provider's flow rather than hedging it, the broker is the counterparty, so broker market P&L on that flow is `neg profit`. And `.brk.notional` (`volume * openPrice`) is a relative exposure measure only. There is no per symbol contract size table in this dataset, so it ranks and shares correctly across symbols and providers but is not a real currency amount.

brokerTech.q - .brk.executed / .brk.orders

```
//@func   | .brk.executed
//@param  | t | table | raw `trade rows
//@desc
// Real fills only: kind=`trade and cancelled=0. Everything downstream
// that talks about P&L / hold time / win rate starts here.
//@desc
.brk.executed:{[t] select from t where kind=`trade, cancelled=0};

//@func   | .brk.orders
//@param  | t | table
//@desc
// All order rows (fills + never-executed cancels), kind=`trade. For
// cancel-rate / order-mix behaviour, where the cancels are the point.
//@desc
.brk.orders:{[t] select from t where kind=`trade};
```

brokerTech.q - .brk.executed and .brk.orders, the fills-only vs fills-plus-cancels split

Two small helper functions, but the split matters. Anything about profit or hold time or win rate has to exclude cancelled orders, because a cancelled order was never a trade. Anything about cancel rate or order mix has to include them, because the cancels are exactly what that question is about. Getting this wrong in either direction quietly poisons every table built on top of it.

Exposure and concentration

The first question a risk desk asks is how much of the book is sitting in one place. `.brk.expo.bySymbol` and `.brk.expo.bySignal` roll up gross notional, buy and sell lots, net lots and client P&L per symbol and per

provider. `.brk.expo.bookConcentration` turns that into a single snapshot using a Herfindahl index on both dimensions, plus top one and top five concentration percentages. Running it against a real 180 day window (2026.03.14 through 2026.09.10, 167 active providers, 106,567 executed trades across 205 symbols) gives:

Live result - book concentration, real 180-day window

metric	val
symbolHHI	0.320316
signalHHI	0.2581388
top1SymbolPct	43.62734
top5SymbolPct	97.13581
top1SignalPct	41.8536
top5SignalPct	91.78883
nSymbols	205
nSignals	167

Live result - book concentration, real 180-day window

Ninety seven percent of book notional sitting in five symbols, out of 205 traded, is a genuinely concentrated book. That single number is why a desk cares about `.brk.expo.peakConcurrent` too, a separate and more interesting measure than a simple notional sum:

brokerTech.q - .brk.expo.peakConcurrent

```
//@func   | .brk.expo.peakConcurrent
//@param  | trades | table
//@desc
// Per symbol: the maximum lots open at the same instant over the window,
// reconstructed from a sweep line over (+volume at openTime, -volume at
// closeTime). A real "peak exposure" number the per-trade rollups can't
// give - a book can trade huge cumulative volume while never holding
// much at once, or vice versa.
//@desc
.brk.expo.peakConcurrent:{[trades]
  t:.brk.executed trades;
  t:select symbol, openTime, closeTime, volume from t
    where not null openTime, not null closeTime, closeTime≥openTime;
  if[not count t;:([] symbol:`symbol$(); peakConcurrentLots:`float$(); peakAt:`timestamp$())];
  ev:(select symbol, tm:openTime, d:volume from t),
     (select symbol, tm:closeTime, d:neg volume from t);
  ev:`symbol`tm xasc ev;
  ev:update run:sums d by symbol from ev;
  `peakConcurrentLots xdesc select
      peakConcurrentLots:max run,
      peakAt:first tm where run=max run
    by symbol from ev};
```

brokerTech.q - .brk.expo.peakConcurrent, a sweep line for peak exposure

That is a proper sweep line: every open becomes a plus event, every close becomes a minus event, sort by time within symbol, take a running sum, and the maximum of that running sum is the true peak concurrent position, not an approximation from average holding size. On this window XRPUSD peaked at 2,100 concurrently open lots on August 22nd, well ahead of anything else in the book. A total volume figure over the same window would never have surfaced that, because most of that symbol's trading could be small and short lived elsewhere in time. This is the same distinction `primeFinance.q`'s crowding module draws between scarcity in a lending market and crowding in the underlying, two related but genuinely different questions that a single blended number would hide.

Execution quality

`.brk.exec.*` looks at how orders actually get placed and closed, not just their outcome. A high cancel rate is a classic quote stuffing or book probing signal, orders placed to test or move price rather than to actually trade. `.brk.exec.orderMix` splits market versus pending orders and limit versus stop within pending. `.brk.exec.holdTimeDist` gives hold time percentiles, the scalping and latency gaming lens. `.brk.exec.stopSlippage` is a proxy for how badly a stopped out trade actually closed relative to its stop level, since the raw feed carries no requested versus filled price pair to measure true slippage against.

brokerTech.q - .brk.exec.stopSlippage

```q
//@func   | .brk.exec.stopSlippage
//@param  | trades | table
//@desc
// Per signal, over trades that carried a stop (sl>0): how often the exit
// landed at or beyond the stop, and the average/max adverse gap between
// the stop level and the actual close. A proxy for stop-out slippage -
// the raw feed has no requested-vs-filled pair, so this is close price
// vs stop level, not a true slippage measurement.
//@desc
.brk.exec.stopSlippage:{[trades]
  t:.brk.executed trades;
  t:select from t where sl>0, not null closePrice, action in `buy`sell;
  if[not count t;:([] signalId:`long$(); nWithStop:`long$(); nStoppedOut:`long$();
    stopHitRatePct:`float$(); avgAdverseGapPx:`float$(); maxAdverseGapPx:`float$())];
  t:update stoppedOut:?[action=`buy; closePrice≤sl; closePrice≥sl],
      gapPx:?[action=`buy; sl-closePrice; closePrice-sl] from t;
  select nWithStop:count i,
      nStoppedOut:sum stoppedOut,
      stopHitRatePct:100*(sum stoppedOut)%count i,
      avgAdverseGapPx:avg (gapPx where stoppedOut and gapPx>0),
      maxAdverseGapPx:max (0f,gapPx where stoppedOut)
    by signalId from t};
```

brokerTech.q - .brk.exec.stopSlippage

This module's own README makes a specific, checkable claim about execution behaviour by broker: that IC Markets ECN accounts show roughly a 2 percent order cancel rate against 55 to 75 percent for pending order heavy books, offered as evidence that the parsed broker rollup gives a real comparison even though most providers name no broker at all. Running `.brk.broker.exec` against the same 180 day window this piece draws its other numbers from does not confirm that:

Live result - cancel rate by broker, real 180-day window

brokerTag	nProviders	nOrders	nExecuted	nCancelled	cancelRatePct
Tickmill	2	6180	783	5397	87.3301
Eightcap	2	779	147	632	81.12965
Exness	1	2337	546	1791	76.63671
Pepperstone	1	1976	497	1479	74.84818
FPMarkets	1	1029	363	666	64.72303
UNKNOWN	152	213507	97476	116031	54.34529
IC Markets	6	12591	6376	6215	49.36065
Fusion	1	263	263	0	0
XM	1	116	116	0	0

Live result - cancel rate by broker, real 180-day window

IC Markets sits at 49.4 percent, not 2 percent, and widening the window to two years brings it to 42.8 percent rather than closer to the claimed figure. The rest of the claim, that pending order heavy books sit well above IC Markets, does hold up (Tickmill at 87.3 percent is a genuinely different execution profile), but the specific 2 percent figure looks like it was written against an older cut of the data, or a different sample, and never rechecked. Worth reporting plainly rather than quietly fixing, the same stance this whole series has taken toward a stale comment or an unverified figure anywhere else in this codebase.

Toxicity scoring

There is no labelled "this account was toxic" training set anywhere in this dataset, so `.brk.tox.score` does not try to fit one. It builds a toxicity score out of five separate, individually explainable components, each on a 0 to 1 scale, weighted and summed:

brokerTech.q - .brk.tox.score (the five-component formula)

```
//@desc
// One row per signal provider with five component scores (each 0..1), a
// weighted composite `toxScore, and its bucket:
//   scalpScore      - sub-5-min share + trades/day (feed scalping)
//   martingaleScore - avg lot-size ratio after a loss vs after a win
//                     (doubling-down / martingale)
//   oneSidedScore   - directional bias, |2*buyShare - 1|
//   burstScore      - share of consecutive opens < burstGapMin apart
//                     (grid / burst entries)
//   tooGoodScore    - high win rate with little/no drawdown (a book
//                     that never loses is usually exploiting stale
//                     pricing or latency, not skill)
// Weights and thresholds are all in .brk.cfg. Deterministic and
// explainable - no labelled training data exists here to fit against.
//@desc
```

brokerTech.q - .brk.tox.score, the five-component formula

Each component reads directly off something a desk already watches individually. Scalping and martingale sizing and one sided directional bias and burst grid entries and an account that simply never seems to lose are all real patterns a risk analyst would flag by eye given enough time looking at blotters. The formula just makes that judgement explicit, weighted, and reproducible, so it can run over the whole book every time rather than over whichever accounts happened to get manual attention. Running `.brk.tox.score` over the same window and bucketing the result gives:

Live result - toxicity bucket counts, real 180-day window

bucket	nProviders
LOW	160
MEDIUM	7

Live result - toxicity bucket counts, real 180-day window

No account crossed into HIGH or EXTREME in this particular window at the minimum ten trade cutoff, which is itself a useful, honest result: the desk's current flow is not, on this deterministic reading, carrying obviously abusive accounts right now. A handful of the MEDIUM accounts are worth a closer look, for example one provider with a martingale score near 1 and a maximum drawdown past minus 35 percent in the same window, exactly the combination the component scores are designed to surface together rather than separately.

A book, B book, and what the routing call is actually worth

Once a provider's toxicity and profitability and size are known, the desk has to decide what to do with that flow. `.brk.book.recommend` turns that decision into an explicit rule rather than a discretionary call:

brokerTech.q - .brk.book.recommend (A/B/SPLIT routing rule)

```
//@desc
// Per signal, a deterministic routing call - route in `A`B`SPLIT with a
// one-line rationale and the expected broker revenue under that route:
//   toxScore ≥ toxHigh            → A  (don't warehouse sharp flow)
//   client net profit > threshold → A  (a consistent winner - hedge)
//   total lots > largeSizeLots    → SPLIT (partial hedge)
//   otherwise                     → B  (losing/neutral non-toxic - internalise)
// SPLIT blends aBookRev / bBookTotalRev by cfg`splitHedgeFrac. Same
// stance as primeFinance's allocator: an explainable rule, swappable for
// an optimiser later without changing the signature.
//@desc
```

brokerTech.q - .brk.book.recommend, the A/B/SPLIT routing rule

Toxic flow gets hedged out regardless of anything else, because warehousing a sharp trader's risk is a bad trade even if that account is currently small. A consistently profitable client gets hedged too, since a client who reliably wins is a client the desk should not want to be the counterparty to. Genuinely large size gets partially hedged rather than fully internalised or fully passed through. Everything else, a losing or roughly neutral account that is not toxic, gets internalised, because that flow is where a B book actually earns money. `.brk.book.optimise` then adds up what the desk would make under all three extremes and under the recommended split:

Live result - A/B routing optimisation headline

metric	val
allBBookRev	-345795f
allABookRev	39263.93
recommendedRev	110748.2
upliftVsBestExtreme	71484.3
nSignals	167
nRouteA	142i
nRouteB	15i
nRouteSplit	10i

Live result - A/B routing optimisation headline

Internalising everything on this book would have lost the desk 345,795 in this window, because enough of this particular provider set are net winners that a pure B book eats their winnings as a cost. Hedging everything through to a liquidity provider earns only the commission line, 39,264, safe but leaves money on the table from the losing accounts that are genuinely profitable to keep in house. The recommended per provider split earns 110,748, an uplift of 71,484 over the better of the two extremes, entirely from routing 142 providers to A, 15 to B, and 10 to a partial SPLIT rather than treating the whole book the same way. That gap between the two naive extremes and the actual per client routing decision is the whole business case for this module existing at all.

Client clustering

The last analytical piece is behavioural, not financial: grouping providers by how they trade rather than by how much money they make. `.brk.cluster.run` builds eight standardised features per provider (average lot size, median hold time, symbol concentration, a three way Asian, London, New York session mix, profit factor, and a size profit bias measuring whether an account wins more on its own bigger trades or its own smaller ones) and runs a from scratch k means over them.

The one design choice worth calling out is the seeding. Ordinary k means seeds its starting centroids randomly, which means two runs over the exact same data can land on different clusters. That is a bad property for a report a desk is going to look at every day and expect to make sense of. `.brk.cluster.priv.initCentroids` instead does a deterministic farthest point traversal: the first centroid is the point farthest from the global mean, then each next centroid is whichever remaining point is farthest from its own nearest already chosen centroid, no randomness anywhere in the namespace. The same window always produces the same clusters, and the same clusters get the same labels, because each cluster's name is derived from its own two most distinctive features rather than hand assigned.

Running it with k equal to five over the same window produces:

Live result - .brk.cluster.run, k=5, real 180-day window

clusterId	nProviders	avgLots	medHoldMin	nSymbols	profitFactor	sizeProfitBias	netProfit	label
4	129	0.153	1214.3	5.12	2.229	15.29	322440.4	London-session
3	28	0.092	620.6	8.96	1.623	69.92	71690.9	NY-session
2	8	5.993	178.0	4.75	3.034	-6.77	107714.3	large size
0	1	1.924	216.0	1.00	0.931	-7681.99	-147851.0	wins small trades, Asian-session
1	1	0.01	99061.0	2.00	1.149	-1.09	89.6	swing/position, Asian-session

Live result - .brk.cluster.run, k=5, real 180-day window

Cluster 4 is the bulk of the book by headcount, 129 small, London hours providers. Cluster 2 is only eight providers but 66,049 of the window's 76,671 total lots, the large size accounts that dominate raw volume without dominating headcount. Cluster 0 is a single provider on its own, and its net profit figure, minus 147,851, is not a coincidence: that is the same account that appeared as the top SPLIT routed provider in the A/B routing detail above, a large loser whose loss is large enough on its own to earn its own cluster rather

than blending into any group average. The clustering did not need to be told that account was unusual. It fell out of the geometry on its own.

Revenue attribution and risk alerts

`.brk.rev.byInstrument` and `.brk.rev.bySignal` answer the plain question of which instruments and which providers actually make the desk money, combining B book market P&L with commission and swap revenue. On this window `EURUSD` alone earned the desk over 22,000 in commission and swap even though its B book market P&L was slightly negative, while `GOLD` lost the desk nearly 81,000 in market risk with almost no offsetting fee revenue, exactly the kind of instrument level breakdown that tells a desk where its hedging policy needs to be stricter regardless of what any individual client's toxicity score says.

`.brk.alert.breaches` turns drawdown, cancel rate, concentration, and toxicity thresholds into one alert feed, the same shape as `primeFinance.q`'s `.prime.alerts`:

brokerTech.q - .brk.alert.breaches

Over the same window it raised 185 breaches: 41 HIGH severity drawdown breaches, 109 MEDIUM concentration breaches, and 35 MEDIUM cancel rate breaches, with alerts gated on a minimum trade or equity sample count so a two point curve cannot raise a spurious breach on noise alone.

The openDash layer: one query, then two live simulators

None of this is useful to a desk sitting in a terminal running `run.q` by hand. `openDash`'s gateway wraps the whole suite in one function, `BrokerTechReader.read`, that pulls the trade and equity window once and runs every `.brk.*` section against it in a single q round trip, cached per parameter set with a time to live so a page polling every twenty seconds does not hammer the HDB:

openDash gateway (brokerTech.js) - echoing .brk.cfg for the frontend

```
summary:.brk.rev.summary w;
revI:.brk.rev.byInstrument w;
bookConc:.brk.expo.bookConcentration w;
peak:.brk.expo.peakConcurrent w;

// deterministic-formula inputs echoed back so the frontend's A/B what-if
// simulator can seed its sliders from the desk's actual current policy
// (.brk.cfg) rather than hardcoding a guess.
thr:`toxHighThreshold`profitableClientUsd`largeSizeLots`splitHedgeFrac!(
  .brk.cfg`toxHigh; .brk.cfg`profitableClientUsd; .brk.cfg`largeSizeLots; .brk.cfg`splitHedgeFrac);
```

openDash gateway (brokerTech.js) - echoing .brk.cfg for the frontend simulator

That comment is the key design decision behind two of the five dashboard pages. `.brk.cfg`, the one dictionary holding every threshold and blend weight in the library, gets echoed straight back in the JSON response alongside the raw per provider fields those formulas were computed from. The A/B Book page seeds four sliders from that echoed policy and then re-implements `.brk.book.recommend`'s decision in plain JavaScript, client side:

openDash (main.jsx) - abSimRoute, the A/B Book page simulator

```
function abSimRoute(r, thr) {
  const tox = r.toxScore ?? 0, cnp = r.clientNetProfit ?? 0, lots = r.totalLots ?? 0;
  if (tox ≥ thr.toxHighThreshold) return { route: "A", rationale: "high toxicity - pass through, do not warehouse" };
  if (cnp > thr.profitableClientUsd) return { route: "A", rationale: "consistently profitable client - hedge to LP" };
  if (lots > thr.largeSizeLots) return { route: "SPLIT", rationale: "large size - partial hedge" };
  return { route: "B", rationale: "losing/neutral non-toxic flow - internalise" };
}
```

openDash (main.jsx) - abSimRoute, the A/B Book page's client-side routing formula

That is the exact same branching logic as the q function it mirrors, translated line for line. Every active provider gets re-scored on every slider tick through a `useMemo`, pure arithmetic against data already in the browser, no network round trip at all. A risk analyst can drag the toxicity cutoff down and watch, instantly, how many providers flip route and what that does to expected revenue, without waiting on kdb+ for every adjustment. The Toxic Analysis page does the same thing for the five component weights and the three bucket cutoffs behind `toxScore` itself, down to reimplementing the composite formula:

openDash (main.jsx) - txSimScore, the Toxic Analysis page simulator

```
function txSimScore(r, w) {
  return (w.wScalp * (r.scalpScore ?? 0)) + (w.wMartingale * (r.martingaleScore ?? 0))
    + (w.wOneSided * (r.oneSidedScore ?? 0)) + (w.wBurst * (r.burstScore ?? 0)) + (w.wTooGood * (r.tooGoodScore ?? 0));
}
```

openDash (main.jsx) - txSimScore, the Toxic Analysis page's client-side formula

Client Clusters cannot take that shortcut, because the cluster count `k` is a genuine clustering parameter, not a display cap the way `topN` is elsewhere. Changing it changes what k means actually get computed, so that page's k selector triggers a real round trip to a second endpoint, `/api/brokertech/clusters`, running `.brk.cluster.run` fresh each time. Execution Quality is the plainest of the five pages, a pure drilldown over `.brk.exec.orderMix` and `.brk.exec.stopSlippage`, two functions that existed in `brokerTech.q` from early on but, per the gateway code's own comment, were "written but unused until now" until this page finally gave them somewhere to be shown.

Known limitations

`monthly.returnPct` is whatever a provider reported to the signals marketplace, and some rows carry absurd figures from cent account bugs or mis-scaled percentages. `.brk.perf.monthlyStats` passes those figures through faithfully rather than filtering them, so that section should be read as what the provider claims, not a computed truth.

Drawdowns worse than minus 100 percent are real in this dataset, an MT account whose equity went negative against a positive peak. That is a genuine blowup worth surfacing, not a calculation error.

`.brk.notional` and every dollar labelled figure in this module are in the dataset's own account currency, uncorrected, since there is no FX feed wired in here to convert across currencies.

Routing is one provider at a time against static window economics. Two simultaneous large requests against the same book are not jointly optimised for a fair split, a first come design choice rather than an optimality guarantee, and the module does not claim otherwise.

The per broker cut only names a broker for providers who put it in their own signal name, a minority of the book, so UNKNOWN is usually the largest single bucket. The named brokers still give a real comparison, as the execution quality section above shows, but that comparison covers only the roughly ten percent of providers who disclosed anything at all. A symbol suffix scheme (`.p`, `.r`, `.fx`, `micro`) is a second, always present proxy for broker identity that is not wired in yet.

Conclusions

None of the individual pieces here, a Herfindahl concentration index, a sweep line for peak exposure, a five component toxicity score, a routing rule, a from scratch k means, are exotic on their own. What makes them worth building together, in one library, over one real dataset, is that each answers a question a risk desk actually asks every day, and each does it as a deterministic, inspectable formula rather than a fitted model, because there is no labelled ground truth anywhere in this data to fit one against. The openDash layer's two live simulators are the natural next step of that same stance: if a formula's weights live in one plain dictionary and get echoed back over the wire, a desk analyst can question and adjust that formula in real time instead of treating it as a black box, exactly the way `.brk.cfg` was designed to be questioned rather than trusted

blindly. And checking the module's own README claim about IC Markets execution quality against a real, live run, rather than repeating it, is the same discipline this whole series keeps coming back to: a formula is only as good as its last check against real data, and a comment describing that data is not exempt from the same check.